

Tokenization Note-Taking Framework

Cole Drumheller

Friday 19th September, 2025 - 02:01

1 Summary

Start with talking about basic tokenization, splitting strings. Word tokens, character tokens, subword tokens. Glitch token section.

This week we are talking about tokenization. Tokenization from a very basic standpoint is just splitting up sentences into individual words. In this case each word would be its own token. This is called word tokenization, we can also tokenize a corpus by characters or sub-words (Tokenization, O'Reilly). The first video talked about glitch tokens, these are tokens that a large language model doesn't recognize properly and therefore doesn't respond correctly. These glitch tokens were caused by how the models were trained, their original training data had these tokens in it, but they were later removed. This causes the model to have these as tokens, and yet not really have any idea what they mean because it was never truly trained on them (Glitch Tokens, Computerphile).

Other aspects of tokenization that were talked about were stopwords, stemming, and lemmatization. These are words that don't really mean anything, but show up a lot. Words such as: 'and', 'the', 'it', 'to', etc. Leaving these words in a training corpus can increase our dimensionality and make it harder for a model to realize the difference between words. Due to this we want to remove these words from our training data (Stopwords, O'Reilly). Stemming is a process to simplify words to their root word by getting rid of the endings. For example turning 'running' into 'run' by dropping the 'ing'. This process is not perfect though and can sometimes cut off too much or too little, leaving you with not a real word. Then we look at lemmatization which has the same goal as stemming, but takes a different approach. This method uses rules of language to return the true root word/meaning of the word. For example, 'better' turns into 'good'. This is a more complex method than stemming (Stemming and Lemmatization, O'Reilly).

Finally, we look at a common tokenization algorithm called Byte-Pair Encoding. Tokenization algorithms have 2 basic parts, a token learner and a token segmenter. A learner takes the training data and makes the set of tokens, then the segmenter takes test sentences and tokenizes them according to the tokens the learner created. The way Byte-Pair works is each character starts as an individual token. Then the tokens are merged together based on which

ones are most commonly adjacent to one another. This is beneficial because it results to frequently used words and sub-words being unique tokens (1 5 Byte Pair Encoding).

2 Attention Grabbers

1. I thought it was interesting how he showed the different tokens that the models divide the prompt into. It doesn't see/read words like we do, instead everything is split into the tokens. (Glitch Tokens - Computerphile, 4:45).
2. It's crazy that these usernames break LLMs. What might be crazier is that it's because they've contributed so much to a counting subreddit. (Glitch Tokens - Computerphile, 15:30).
3. Subword tokenization was interesting because I've thought about how words that are almost the same but are different like the walk vs walking example would be seen as similar. With enough training the model would hopefully identify that they are similar, but subword tokenization seems like a much better way to do it.
4. At the end of the tokenization vidoe the guy talks about how tokenization is not machine learning. It's just basic coding as he called it, which I thought was interesting because tokenization seems like one of those things that just wouldn't be as simple as it actually is.
5. I was wondering how subword tokenization would work. The Byte-Pair algorithm's approach of tokenizing each character and then grouping ones that are most commonly together makes a lot of sense.

3 Questions

1. How can we ensure that we don't cause glitch tokens when training a model?
2. For word tokens vs subword tokens, is it more beneficial to use full words if you have enough data to train on?
3. When would it be beneficial to do character tokenization?
4. Is lemmatization computationally expensive? It seems like a somewhat complex process.
5. Is the best tokenization strategy/algorithm decided just by trial and error or are there ways to tell which will be best for a given corpus?